

AI Multi-Object Detection System

Project README — YOLOv8 / OpenCV / PyTorch

An AI-powered multi-object detection system built using YOLOv8, OpenCV, and PyTorch. This project was developed as part of an internship project at Hindalco Industries Limited, and detects and tracks multiple objects in real time from images, videos, or live camera feeds.

Note: Descriptions for `main.py`, `dd.py`, `tt.py`, and `convert_xml_to_yolo.py` are inferred from standard YOLOv8 project conventions and typical file naming, since only the file/folder structure (not code content) was available while writing this document. Please verify against your actual code and edit as needed.

Project Structure

```
AI MULTI OBJECT DETECTION/
|-- .venv/                                # Python virtual environment (auto-generated)
|   |-- Include/
|   |-- Lib/
|   |-- Scripts/
|   |-- .gitignore
|   |-- pyvenv.cfg
|-- dataset/                             # Training/validation/testing data
|   |-- test/                            # Test images + labels
|   |-- train/                           # Training images + labels
|   |-- valid/                           # Validation images + labels
|   |-- data.yaml                         # Dataset configuration file (classes, paths)
|-- outputs/                             # Saved output files (images/videos)
|-- runs/                                # YOLO auto training/inference logs
|-- venv/                                # (Possible duplicate) venv folder
|-- videos/                              # Input videos for testing/inference
|-- .gitattributes                        # Git config for file types (LFS etc.)
|-- convert_xml_to_yolo.py                # Converts XML annotations to YOLO .txt format
|-- dd.py                                # Custom detection/data-processing script
|-- main.py                              # Main entry point - runs the detection pipeline
|-- output.avi                           # Sample output video (AVI) from a detection run
|-- output.mp4                           # Sample output video (MP4) from a detection run
|-- requirements.txt                      # Python dependencies needed to run the project
|-- tt.py                                # Test/training helper script
|-- yolo26n.pt                            # Trained/pretrained YOLO model weight file
|-- yolov8m.pt                            # YOLOv8 'medium' weight file
|-- yolov8n.pt                            # YOLOv8 'nano' weight file (fastest)
|-- yolov8s.pt                            # YOLOv8 'small' weight file
```

File-by-File Explanation

`main.py`

This is the main script of the project — the entry point you run to start object detection. It typically:

- Loads a trained YOLO model (.pt file)
- Loads an input source (image, video, or webcam)
- Runs inference (detection) frame-by-frame
- Draws bounding boxes + labels on detected objects
- Saves or displays the output

Run it with:

```
python main.py
```

convert_xml_to_yolo.py

A data preparation utility. If your dataset was originally annotated in XML format (e.g., using tools like Labellmg in Pascal VOC format), this script converts those XML annotation files into YOLO .txt label format (class_id x_center y_center width height, normalized between 0-1), which is what YOLOv8 requires for training.

You'd use this script once, right after collecting/annotating a new dataset in XML, before training.

dd.py

Appears to be a helper/detection script — possibly used for testing detection on a specific image/video, debugging model outputs, or a draft/duplicate version of the detection logic. Recommend renaming this to something descriptive (e.g. detect_debug.py) for clarity, or removing it if unused.

tt.py

Likely a training or testing script — possibly used to train the YOLO model on your dataset (model.train(...)) or to run quick test inferences. Recommend a clearer filename such as train.py or test.py.

requirements.txt

Lists all the Python packages this project depends on (e.g. ultralytics, opencv-python, torch, numpy, etc.). Install them all at once with:

```
pip install -r requirements.txt
```

output.avi / output.mp4

Sample output videos generated after running detection on an input video — showing bounding boxes and labels drawn on detected objects. These are results, not code — safe to delete/regenerate anytime.

.gitattributes

A Git configuration file that tells Git how to handle certain file types (e.g., normalize line endings, treat .pt model files as binary/LFS). Doesn't affect how the code runs — only version control behavior.

Model weight files (.pt)

These are trained YOLO model files (PyTorch format):

File	Meaning
------	---------

yolov8n.pt	YOLOv8 Nano — smallest & fastest, lower accuracy
yolov8s.pt	YOLOv8 Small — balance of speed & accuracy
yolov8m.pt	YOLOv8 Medium — more accurate, slower
yolo26n.pt	Likely a custom-trained model (or newer YOLO version — verify naming)

main.py loads one of these to perform detection. You can swap between them depending on your speed vs. accuracy needs.

The dataset/ Folder

```
dataset/
|-- train/      -> Images + labels used to train the model
|-- valid/      -> Images + labels used to validate the model during training
|-- test/       -> Images + labels used to test the final model
`-- data.yaml   -> Tells YOLO where the above folders are, and what classes exist
```

Each of train/, valid/, and test/ usually contains two subfolders internally:

```
train/
|-- images/     # .jpg / .png files
`-- labels/     # .txt files (one per image, YOLO format annotations)
```

Example data.yaml

```
train: dataset/train/images
val: dataset/valid/images
test: dataset/test/images

nc: 3                # number of classes
names: ['person', 'car', 'bike'] # class names, in order matching class IDs
```

How to Change / Replace the Dataset

Follow these steps if you want to train the model on a new or different dataset:

Step 1: Collect & Annotate New Images

- Gather your new images.
- Annotate each object with a bounding box using a tool like LabelImg, Roboflow, or CVAT.
- Export annotations in YOLO format directly if possible. If you only have XML (Pascal VOC) annotations, continue to Step 2.

Step 2: Convert Annotations (if needed)

If your new annotations are in XML format:

```
python convert_xml_to_yolo.py
```

This generates .txt label files in YOLO format for each image. Check inside the script for the exact input/output folder paths it expects — update them if your new data is in a different location.

Step 3: Organize Folder Structure

Replace the contents of the dataset/ folder with your new data, following this exact structure:

```
dataset/
|-- train/
|   |-- images/    <- ~70-80% of your new images
|   |-- labels/    <- matching YOLO .txt files
|-- valid/
|   |-- images/    <- ~10-15% of images, for validation
|   |-- labels/
|-- test/
|   |-- images/    <- ~10-15% of images, for final testing
|   |-- labels/
`-- data.yaml
```

Each image file must have a matching .txt label file with the same filename (e.g. img001.jpg <-> img001.txt).

Step 4: Update data.yaml

Edit dataset/data.yaml to reflect:

- The correct folder paths (if changed)
- The number of classes (nc)
- The list of class names, in the exact order matching the class IDs used in your labels

Example — if your new dataset has 5 classes instead of 3:

```
train: dataset/train/images
val: dataset/valid/images
test: dataset/test/images

nc: 5
names: ['helmet', 'vest', 'gloves', 'boots', 'goggles']
```

Step 5: Retrain the Model

Use the Ultralytics YOLO training command (in tt.py, or directly in a new script/terminal):

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt') # start from a pretrained base model
model.train(data='dataset/data.yaml', epochs=100, imgsz=640)
```

Or from the command line:

```
yolo task=detect mode=train model=yolov8n.pt data=dataset/data.yaml epochs=100
imgsz=640
```

Another common variant — using the Small model, a lower batch size (useful on GPUs with less VRAM), and pinning the run to a specific GPU/worker count:

```
yolo detect train model=yolov8s.pt data=data.yaml epochs=75 imgsz=640 batch=4
device=0 workers=4
```

- model=yolov8s.pt — starts training from the Small pretrained checkpoint instead of Nano (more accurate, slightly slower).
- epochs=75 — runs 75 full passes over the training data.
- batch=4 — processes 4 images per training step; lower batch sizes use less GPU memory, which helps on GPUs with limited VRAM (e.g. 4–6 GB cards).

- `device=0` — trains on GPU index 0 (the first CUDA-capable GPU). Use `device='cpu'` to force CPU training instead.
- `workers=4` — uses 4 CPU worker processes to load and augment images in parallel, keeping the GPU fed with data.

Note: If you hit a CUDA "out of memory" error during training, lowering batch further (e.g. `batch=2`) is usually the quickest fix, followed by reducing `imgsz`.

This creates a new folder inside `runs/detect/train/` containing:

- `weights/best.pt` -> your newly trained model (use this in `main.py` going forward)
- `weights/last.pt` -> the last checkpoint
- Training graphs, confusion matrix, sample predictions, etc.

Step 6: Update `main.py` to Use the New Model

In `main.py`, change the model path from an old `.pt` file to your newly trained one:

```
model = YOLO('runs/detect/train/weights/best.pt')
```

Step 7: Run Detection

```
python main.py
```

Installation & Setup

1. Clone/download the project.
2. Create and activate a virtual environment (see below).
3. Install dependencies with `pip install -r requirements.txt`
4. Run the detection system with `python main.py`

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS/Linux

pip install -r requirements.txt
python main.py
```

Housekeeping Notes

- You currently have two virtual environment folders (`.venv/` and `venv/`) — consider removing one to avoid confusion and save space.
- `__pycache__/` contains compiled Python bytecode (`.pyc` files) — safe to delete anytime, Python regenerates it automatically.
- Consider adding a `.gitignore` to exclude `.venv/`, `venv/`, `__pycache__/`, `runs/`, and `outputs/` from version control, since these are large/auto-generated.

Moving This Project to a New PC

If you're setting this project up on a different computer, the short answer is: copy everything except the `venv` folder, and create a fresh virtual environment on the new machine. Virtual environments contain absolute paths and OS-specific binaries, so a `venv/.venv` folder copied from one PC almost never works correctly on another — always recreate it instead.

Step 1: Copy the Project (Without the Virtual Environment)

Copy your whole project folder to the new PC, but leave `.venv/` and `venv/` behind. Your copied folder should look something like this:

```
AI-Multi-Object-Detection/  
|-- main.py  
|-- requirements.txt  
|-- yolov8n.pt / best.pt  
|-- dataset/  
|-- videos/  
|-- outputs/  
|-- .venv/      <- do NOT copy this  
`-- venv/       <- do NOT copy this
```

Step 2: Install Python on the New PC

Make sure the same (or a compatible) Python version is installed as on your original machine. Check the installed version with:

```
python --version
```

Step 3: Create a New Virtual Environment

On the new PC, create a fresh virtual environment instead of reusing the old one:

```
python -m venv venv
```

Step 4: Activate the Virtual Environment

Command Prompt:

```
venv\Scripts\activate
```

PowerShell:

```
.\venv\Scripts\Activate.ps1
```

macOS/Linux:

```
source venv/bin/activate
```

Step 5: Install All Required Packages

If you already have a `requirements.txt` (you do — see the Requirements section above), install everything from it in one command:

```
pip install -r requirements.txt
```

If you ever need to regenerate requirements.txt from your current working setup (e.g., before moving to a new PC), run this on the original machine and copy the resulting file over:

```
pip freeze > requirements.txt
```

Step 6: Install Ultralytics for YOLOv8

Since this project uses YOLOv8, make sure the Ultralytics package is installed — it's already included in the requirements.txt from earlier in this document, but you can also install it directly:

```
pip install ultralytics
```

Step 7: GPU Setup (If the New PC Has an NVIDIA GPU)

If the new machine has a CUDA-capable NVIDIA GPU, install the CUDA-enabled build of PyTorch instead of the default CPU-only build, so detection runs much faster. For example, for CUDA 12.8:

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu128
```

Note: Use the index URL that matches the CUDA runtime actually installed on the new PC — check <https://pytorch.org/get-started/locally/> for the correct command if the version differs.

Verify that PyTorch can see the GPU:

```
import torch  
print(torch.cuda.is_available())
```

This should print True. If it prints False, PyTorch is falling back to CPU — double-check the CUDA version and that you installed the matching PyTorch build.

Checklist: What to Copy vs. What to Leave Behind

Copy to new PC?	Item
Yes	Your project folder (all source files)
Yes	Trained model weights (e.g. best.pt, yolov8n.pt)
Yes	requirements.txt
Yes	Test images/videos, if needed for testing
No	.venv/ or venv/ folder
No	__pycache__/
No	.pytest_cache/
No	runs/ (unless you specifically want to keep old training results)

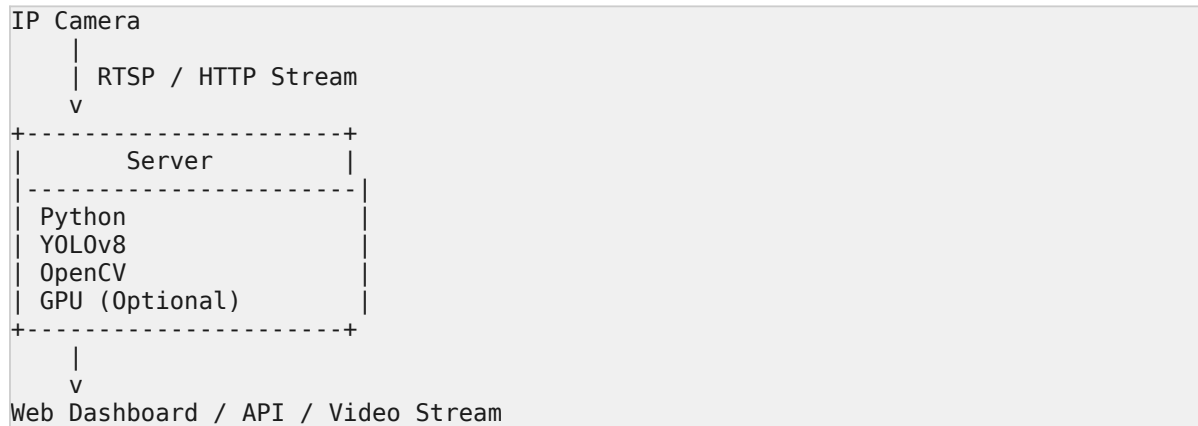
Want a One-Click Version Instead?

Since this is a YOLOv8 multi-object detection system, it's also possible to package it into a single standalone .exe using a tool like PyInstaller or Nuitka. That would let it run on another PC with one click — no Python install, no virtual environment activation, and no manual pip installs required for the person running it. If that would be useful, let me know and I can walk through packaging main.py (and the model weights) into a distributable executable.

Deploying on a Server & Connecting an IP Camera

Running this project on a server is different from running it on your laptop — especially once an IP camera (CCTV/RTSP feed) is involved instead of a local webcam. This section covers the full workflow: setting the project up on a server, swapping the video source for an IP camera stream, and keeping the service running continuously.

Architecture Overview



The server does all of the AI processing. Cameras just stream video in; users just view results out — nobody else needs YOLO or Python installed on their own machine.

Step 1: Copy Your Project to the Server

Copy the complete project folder to the server, the same way as moving to a new PC (see the section above). As before, don't copy `.venv/venv` — create a fresh one on the server.

```
AI-Multi-Object-Detection/
|-- best.pt
|-- app.py
|-- detect.py
|-- requirements.txt
|-- data.yaml
|-- models/
|-- static/
|-- templates/
-- venv/          <- don't copy this
```

Step 2: Install Python

Install the same Python version on the server as you used in development (e.g. Python 3.11).

Step 3: Create a Virtual Environment

Linux:

```
python3 -m venv venv
source venv/bin/activate
```

Windows Server:


```
python -m venv venv
venv\Scripts\activate
```

Step 4: Install Packages

If you have a requirements.txt (you do — see earlier in this document):

```
pip install -r requirements.txt
```

Otherwise, install the core packages individually:

```
pip install ultralytics
pip install opencv-python
pip install flask
pip install numpy
pip install torch torchvision
```

Step 5: Copy Your Trained Model

Copy your trained weights file (best.pt) into the project on the server, e.g. into a weights/ folder:

```
weights/
  best.pt
```

Step 6: Test the Model Locally on the Server

Before wiring up the camera, confirm the app runs correctly on the server itself:

```
python app.py
# or
python detect.py
```

If it works locally on the server, continue to connecting the camera.

Step 7: Connect an IP Camera

Instead of opening a local webcam:

```
cap = cv2.VideoCapture(0)
```

point OpenCV at the camera's network stream instead:

```
cap = cv2.VideoCapture("rtsp://username:password@192.168.1.100:554/stream")
```

or, for HTTP-based streams:

```
cap = cv2.VideoCapture("http://192.168.1.100:8080/video")
```

Common RTSP URL formats by brand:

Camera / Source	Example URL
Hikvision	rtsp://admin:12345@192.168.1.20:554/Streaming/Channels/101
Dahua	rtsp://admin:12345@192.168.1.30:554/cam/realmonitor?channel=1&subtype=0
Android IP Webcam app	http://192.168.1.50:8080/video

Note: Replace the username, password, and IP address with your camera's actual credentials and network address. Most IP cameras expose their exact RTSP path in their admin settings page or manual.

Step 8: Run Detection on the Camera Stream

Once connected, the detection loop works the same way as with a webcam — only the VideoCapture source changes:

```
from ultralytics import YOLO
import cv2

model = YOLO("best.pt")

cap = cv2.VideoCapture("rtsp://admin:1234@192.168.1.20:554/Streaming/Channels/101")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame)
    annotated = results[0].plot()

    cv2.imshow("Detection", annotated)

    if cv2.waitKey(1) == 27:
        break
```

Step 9: Make It Accessible Over the Network

If you're serving results through Flask, bind to all network interfaces (not just localhost) so other devices on the network can reach it:

```
app.run(host="0.0.0.0", port=5000)
```

Anyone on the same network can then open:

```
http://SERVER_IP:5000
```

Step 10: Keep It Running Continuously

For production use, the detection service should restart automatically if it crashes or the server reboots. Common options:

- Linux: systemd (recommended) or Supervisor
- Cross-platform / containerized: Docker
- Windows: Task Scheduler or NSSM (Non-Sucking Service Manager)

Step 11: GPU Acceleration on the Server

If the server has an NVIDIA GPU, install the NVIDIA driver, the CUDA Toolkit, and a CUDA-compatible build of PyTorch, then verify GPU access:

```
import torch
print(torch.cuda.is_available())
```

This should print True. YOLO will then use the GPU automatically, or you can force it explicitly:

```
model.predict(source=frame, device=0)
```

Step 12: Handling Multiple Cameras

Multiple IP camera streams can be opened simultaneously, one VideoCapture object per camera:

```
cameras = [  
    "rtsp://camera1",  
    "rtsp://camera2",  
    "rtsp://camera3"  
]
```

For good performance, process each camera's stream in its own thread or separate process, rather than reading all of them sequentially in a single loop.

Recommended Deployment for This Project

Component	Recommendation
Server OS	Ubuntu Server 22.04 (well-suited for AI workloads)
Python	Same version used in development
Environment	Fresh venv, dependencies installed from requirements.txt
Model	Trained best.pt copied to the server
Camera Input	IP camera RTSP stream instead of cv2.VideoCapture(0)
Backend	Flask, for a web interface or API
Inference	YOLOv8 on the server, GPU-accelerated if available
Production Uptime	Process manager (e.g. systemd) for automatic restarts

Note: For an industrial CCTV monitoring use case like this project's Hindalco context, this is the standard architecture: IP cameras stream video to a central server, the server runs YOLO inference, and users view results through a web dashboard — without needing YOLO, PyTorch, or any AI model installed on their own machines.

Tech Stack

- Python
- YOLOv8 (Ultralytics)
- OpenCV — video/image processing
- PyTorch — deep learning backend

README generated based on project file structure. Please review script descriptions (main.py, dd.py, tt.py, convert_xml_to_yolo.py) against your actual code and adjust any details that don't match.